✅ Integrated: Codon-Contrast + Genomic AGI Immune System Analogy

The **Codon-Contrast biophysical mapping** fits *beautifully* into the **Genomic Analogy of an AGI Immune System** framework from the new document. Here's how they connect at the architectural level, followed by the refactored, enhanced class.

Conceptual Integration

Component	Codon-Contrast Role	AGI Immune System Analog (from document)
6-bit Biophysical Vector	`[GC1, PUR1, GC2, PUR2, GC3, PUR3]`	MHC antigen presentation groove — shape + chemistry binding check (Gate 1: Lipschitz envelope)
Delta Vector Δh	Net codon substitution counts	Foreign "antigen" proposal entering the system
FWHT Contrasts	Orthogonal biophysical features	Independent verification pathways (innate vs adaptive; Gate 1 vs Gate 2 separation)
L1-Normalized Features	Scale-free contrast signature	Self/non-self discrimination (Gate 3) + memory retention
Flow Metrics (added below)	Transition bias, GC drift, entropy	Cytokine rate limiting (Gate 4) + somatic hypermutation monitoring (ADR-004)

This makes **CodonContrastAnalyzer** a perfect **molecular-level sensor** inside the cognitive immune system:

- Detects "pathogenic" codon-level edits (mutations).
- Provides orthogonal features for the four-gate predicate.
- Feeds into L0 innate checks (fast) and L1 adaptive refinement.

Enhanced Class-Based Implementation

File: `/home/workdir/codon_contrast_immune.py`

```
Python

import numpy as np
from typing import Dict, Tuple, List, Optional
import json
```

```
BASE_GC = {'A': 0, 'T': 0, 'C': 1, 'G': 1}
BASE_PUR = {'A': 1, 'G': 1, 'C': 0, 'T': 0}


class CodonContrastAnalyzer:
    """Codon-Contrast biophysical mapper integrated with AGI Immune System
analogy."""

    def __init__(self):
        self.codon_map: Dict[str, Tuple[int, ...]] =
self._generate_codon_mapping()
        self.codon_list: List[str] = sorted(self.codon_map.keys())
        self.codon_to_index: Dict[str, int] = {codon: i for i, codon in
enumerate(self.codon_list)}

    def _generate_codon_mapping(self) -> Dict[str, Tuple[int, ...]]:
        bases = ['A', 'C', 'G', 'T']
        mapping = {}
        for b1 in bases:
            for b2 in bases:
                for b3 in bases:
                    codon = b1 + b2 + b3
                    mapping[codon] = self._codon_to_bits(codon)
        return mapping

    def _codon_to_bits(self, codon: str) -> Tuple[int, ...]:
        codon = codon.upper()
        return (
            BASE_GC[codon[0]], BASE_PUR[codon[0]],
            BASE_GC[codon[1]], BASE_PUR[codon[1]],
            BASE_GC[codon[2]], BASE_PUR[codon[2]]
        )

    def get_biophysical_bits(self, codon: str) -> Tuple[int, ...]:
        return self.codon_map[codon.upper()]

    def compute_delta_h(self, counts_from: Dict[str, int], counts_to:
Dict[str, int]) -> np.ndarray:
        delta = np.zeros(64, dtype=int)
        all_codons = set(counts_from.keys()) | set(counts_to.keys())
        for codon in all_codons:
            idx = self.codon_to_index[codon.upper()]
```

```
            delta[idx] = counts_to.get(codon, 0) - counts_from.get(codon,
0)
        return delta

    @staticmethod
    def fast_walsh_hadamard_transform(x: np.ndarray) -> np.ndarray:
        n = len(x)
        h = np.copy(x.astype(float))
        bit = 1
        while bit < n:
            for i in range(0, n, bit * 2):
                for j in range(i, i + bit):
                    u, v = h[j], h[j + bit]
                    h[j] = u + v
                    h[j + bit] = u - v
            bit *= 2
        return h / np.sqrt(n)

    def compute_biophysical_contrasts(self, delta_h: np.ndarray) ->
np.ndarray:
        fwht_coeffs = self.fast_walsh_hadamard_transform(delta_h)
        l1_norm = np.sum(np.abs(fwht_coeffs))
        return fwht_coeffs / l1_norm if l1_norm > 0 else fwht_coeffs

    def compute_flow_metrics(self, delta_h: np.ndarray) -> Dict[str,
float]:
        """Flow metrics for immune monitoring (Gate 4 + ADR-004)."""
        total_subs = np.sum(np.abs(delta_h))
        if total_subs == 0:
            return {'transition_ratio': 0.0, 'gc_bias': 0.0,
'wobble_fraction': 0.0, 'flow_entropy': 0.0}

        # Simplified metrics (expandable)
        transition_ratio = 0.5  # placeholder - implement full
transition/transversion logic
        gc_bias = np.sum(delta_h * np.array([b[0] + b[2] + b[4] - 1.5 for
b in self.codon_map.values()])) / total_subs
        flow_entropy = -np.sum((np.abs(delta_h)/total_subs) *
np.log2(np.abs(delta_h)/total_subs + 1e-10))

        return {
            'transition_ratio': transition_ratio,
            'gc_bias': float(gc_bias)
```

```
            'gc_bias': float(gc_bias),
            'wobble_fraction': 0.33,  # placeholder
            'flow_entropy': float(flow_entropy)
        }

    def immune_analyze(self, from_seq: Dict[str, int], to_seq: Dict[str,
int]) -> Dict:
        """Full immune-system-style analysis."""
        delta_h = self.compute_delta_h(from_seq, to_seq)
        contrasts = self.compute_biophysical_contrasts(delta_h)
        flow = self.compute_flow_metrics(delta_h)

        top_features = self._get_top_features(contrasts)

        # Immune decision signals
        envelope_safe = np.max(np.abs(contrasts)) < 0.8  # example
Lipschitz-like bound
        contracting = np.max(contrasts) > -0.1           # example
contraction check

        return {
            'delta_h': delta_h.tolist(),
            'contrasts': contrasts.tolist(),
            'top_features': top_features,
            'flow_metrics': flow,
            'immune_status': {
                'envelope_safe': bool(envelope_safe),
                'contracting': bool(contracting),
                'cytokine_storm_risk': envelope_safe and not contracting,
                'self_non_self_score':
float(np.mean(np.abs(contrasts[:6])))  # main effects
            }
        }

    def _get_top_features(self, contrasts: np.ndarray, top_k: int = 5) ->
List[Dict]:
        abs_coeffs = np.abs(contrasts)
        top_indices = np.argsort(abs_coeffs)[-top_k:][::-1]
        return [{'index': int(i), 'value': float(contrasts[i]),
'description': f'Hadamard_{i}'} for i in top_indices]

    def to_json(self) -> str:
        return json.dumps({'codon_map': {k: list(v) for k, v in
```

```
self.codon_map.items()}}, indent=2)


# Demo
if __name__ == "__main__":
    analyzer = CodonContrastAnalyzer()

    # Example: Simulate a "pathogenic" edit
    result = analyzer.immune_analyze(
        {'AAA': 10, 'AAC': 5},
        {'AAA': 9, 'ACA': 1, 'ACC': 5}
    )

    print("🧬 Codon-Contrast AGI Immune Analysis")
    print("=" * 50)
    print("Immune Status:", result['immune_status'])
    print("Top Biophysical Features:", result['top_features'][:3])
    print("Flow Metrics:", result['flow_metrics'])
```

This class now directly supports the **four-gate predicate** and **cytokine storm detection** logic from the immune analogy document.

Would you like me to:

1. Expand the flow metrics with full transition/transversion logic?
2. Add PIRTM/MOC-style RSL truth vector output?
3. Generate a test suite with known "antigen" substitutions?
4. Connect this to the Codon-Contrast PDF validation example?

Let me know the next layer.